

Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 3: Integridad y Restricciones

3.1.3 Semana 1 - Día 3: Integridad y Restricciones

Ahora que los datos están en el motor, entraremos en la fase de **Blindaje de Datos**. El **Día 3** se trata de pasar de una "**tabla que acepta todo**" a una **Estructura de Datos Robusta** que garantice que ningún análisis futuro falle por datos corruptos o inconsistentes.

"El Blindaje del Dato: Constraints y Casting de Tipos"

En la ingeniería de datos moderna (2026), la calidad no es un evento posterior, es una **restricción estructural**. Hoy aprenderás a implementar reglas que impidan el paso de "**basura**" a tu sistema.

Diccionario Técnico Operativo

- **'Constraints' (Restricciones):**
 - Son **reglas predefinidas** que se aplican a **columnas** o **tablas** de una **base de datos** para **limitar los datos** que pueden almacenarse.

```
/*****  
*****
```

Título: Aseguramiento de Integridad de Datos - Tabla de Empleados

Objetivo: Garantizar que los datos de la tabla 'empleados' sean coherentes, validando reglas de negocio básicas antes de guardar la información.

Descripción: Se añaden restricciones (constraints) de tipo CHECK para asegurar que la edad y el salario cumplan con los mínimos establecidos.

```
*****  
*****/
```

```
-- 1. Asegurar que la edad mínima sea 18 años  
-- Objetivo: Evitar la contratación de menores de edad.  
-- Descripción: La columna 'edad' no podrá tener valores inferiores a 18.
```

ALTER TABLE empleados

```
ADD CONSTRAINT chk_edad_minima  
CHECK (edad >= 18);
```

-- 2. Asegurar que el salario no sea negativo
-- Objetivo: Garantizar la integridad financiera de la tabla.
-- Descripción: La columna 'salario' debe ser mayor o igual a cero.

```
ALTER TABLE empleados  
ADD CONSTRAINT chk_salario_positivo  
CHECK (salario >= 0);
```

-- 3. Validación compuesta: Edad debe ser mayor o igual a 18 Y salario > 0

-- Objetivo: Validar múltiples reglas en una sola operación.
-- Descripción: Se comprueba que no haya empleados menores de 18 ni con salario cero o negativo.

```
ALTER TABLE empleados  
ADD CONSTRAINT chk_empleado_valido  
CHECK (edad >= 18 AND salario > 0);
```

- Su **objetivo principal** es **garantizar la integridad, precisión y fiabilidad de la información**; si una acción (como **insertar** o **actualizar** un dato) **viola una regla**, el sistema **cancela la operación automáticamente**.
- **Caso Práctico**: Asegurar que el **monthly_income** nunca sea un **número negativo** en el **dataset de IBM**.
- **Pitfall**: Olvidar que las **restricciones existentes bloquean cargas masivas** si el origen tiene errores (siempre **limpiar** antes de aplicar).
- **Principales Tipos de Constraints (Restricciones)**:
 - **NOT NULL**:
 - Asegura que una **columna** NO acepte **valores nulos (vacíos)**.
 - **Caso Práctico**: Asegurar que cada **empleado** tenga un **employee_id**, **job_role** (Rol laboral) y **monthly_income** (Salario Mensual) para calcular las **tasas de desgaste**.

```
/*****  
*****/
```

Título: Creación de la tabla de datos limpios de rotación

Objetivo: Estructurar una tabla base para almacenar información depurada de los empleados, enfocada en el análisis de rotación (attrition).

Descripción: Este script define la tabla 'attrition_data_clean', estableciendo las columnas necesarias y los tipos de datos para asegurar la integridad de la información (como IDs únicos y roles obligatorios).

```
*****  
*****/
```

```
CREATE TABLE attrition_data_clean(  
    -- Identificador único del empleado, no puede estar vacío  
    employee_id INT NOT NULL,  
    -- Edad del empleado  
    age INT,  
    -- Indica si el empleado se fue (Yes/No)  
    attrition VARCHAR(10),  
    -- Puesto de trabajo, campo obligatorio  
    job_role VARCHAR(100) NOT NULL,  
    -- Salario mensual, obligatorio para análisis económico  
    monthly_income INT NOT NULL,  
    -- Área de estudio o formación académica  
    education_field VARCHAR(100)  
);
```

- **Pitfall:** Aplicar **NOT NULL** en una **tabla** que ya tiene **nulos** sin antes haberlos "imputado" o borrado.
- **UNIQUE:**
 - **Garantiza** que **todos** los **valores** en una **columna** sean **distintos** entre sí.
 - **Caso Práctico:** Asegurar que la columna **employee_number** sea única.

```
/*****  
*****
```

Título: Creación de tabla de empleados de IBM

Objetivo: Estructurar una tabla base para almacenar información clave de los empleados de IBM con fines de análisis de recursos humanos.

Descripción: Este script define una tabla llamada 'ibm_employees' para registrar el número de empleado, edad, departamento y nivel de satisfacción laboral, asegurando que cada empleado sea único.

```
*****  
*****/
```

```
CREATE TABLE ibm_employees(  
    -- Identificador único del empleado (Número de empleado),  
    no puede estar vacío  
    employee_number INT NOT NULL,  
    -- Edad del empleado (tipo entero)  
    age INT,  
    -- Departamento al que pertenece (texto hasta 50 caracteres)  
    department VARCHAR(50),  
    -- Nivel de satisfacción laboral (tipo entero, ej. escala 1-4)  
    job_satisfaction INT,  
    -- Restricción: Asegura que no se repita el 'employee_number'  
    en la tabla  
    UNIQUE(employee_number)  
);
```

- **Pitfall:** Si tiene una columna **phone_number** como **UNIQUE**, puedes insertar varios **NULL**. La **base de datos** no lo considera un **duplicado** porque **NULL** significa "**valor desconocido**", no que sean iguales.

```
/*****  
*****
```

Título: Creación y Prueba de Tabla de Contactos de Empleados

Objetivo: Demostrar el comportamiento de la restricción UNIQUE con valores NULL.

Descripción: El script crea una tabla para almacenar teléfonos de empleados, asegurando que no haya números duplicados, pero permitiendo que un empleado no tenga teléfono registrado (NULL).

```
*****  
*****/
```

-- Bloque 1: Definición de la estructura de la tabla

```
-- Creamos la tabla 'employee_contact' para relacionar IDs con
teléfonos.
CREATE TABLE employee_contact(
    employee_id INT,          -- Identificador único del empleado
    phone_number VARCHAR(20) UNIQUE -- Número de teléfono,
                                la restricción UNIQUE
                                -- evita duplicados, pero permite múltiples NULLs.
);

-- Bloque 2: Pruebas de inserción exitosas (Comportamiento de
NULL)
-- Insertamos registros con teléfono NULL. Esto es correcto
porque SQL
-- considera que NULL no es igual a otro NULL (no es un valor, es
"desconocido").
INSERT INTO employee_contact(employee_id, phone_number)
VALUES(1, NULL);
INSERT INTO employee_contact(employee_id, phone_number)
VALUES(2, NULL);

-- Bloque 3: Pruebas de inserción con error (Duplicados)
-- Este primer INSERT funciona correctamente al agregar un
teléfono.
INSERT INTO employee_contact(employee_id, phone_number)
VALUES(1, '555-1234');

-- Este segundo INSERT fallará (lanzará un error de violación de
restricción)
-- porque el número '555-1234' ya existe para el empleado 1.
INSERT INTO employee_contact(employee_id, phone_number)
VALUES(2, '555-1234');
```

○ **PRIMARY KEY:**

- **Identificador único** para cada **fila** (es una **combinación** de **NOT NULL** + **UNIQUE**).
- **Unicidad**: No puede haber 2 **filas** con el mismo valor en la **columna primaria**.
- **No Nulo (NOT NULL)**: La **columna primaria** no puede contener valores en blanco o nulos.
- **Única por Tabla**: Solo se puede definir un **PRIMARY KEY** por **tabla**.

- **Índice Automático:** SQL crea automáticamente un **índice** para acelerar las **consultas** basadas en esta clave.
- **Caso Práctico:** La columna employee_number o employee_id es ideal para ser PRIMARY KEY, ya que cada registro del empleado debe ser único para evitar duplicar información.

```

/*****
****

```

Título: Creación de Tabla Maestra de Analítica RRHH (hr_analytics)

Objetivo: Estructurar una tabla base para almacenar datos clave de los empleados.

Descripción: Este script crea la tabla 'hr_analytics' para centralizar información demográfica, de satisfacción y económica, permitiendo realizar estudios de rotación de personal (attrition).

```

****
*****/

```

-- Se crea la tabla 'hr_analytics' para almacenar la información de los empleados

```

CREATE TABLE hr_analytics(
    -- ID único del empleado. No puede ser nulo, funciona como
    -- identificador principal.
    employee_number INT NOT NULL,
    -- Edad del empleado (para análisis demográficos).
    age INT,
    -- Indica si el empleado dejó la empresa ('Yes'/'No'). Campo
    -- clave para modelos de predicción.
    attrition VARCHAR(10),
    -- Nivel de satisfacción laboral (ej. escala del 1 al 4).
    job_satisfaction INT,
    -- Salario mensual del empleado.
    monthly_income INT,
    -- Define 'employee_number' como clave primaria para
    -- asegurar que cada empleado sea único.
    PRIMARY KEY(employee_number)
);

```

- **Pitfall:**

- **Intentar usar NULL:** Definir una **columna** con **valores nulos** como **PRIMARY KEY** provocará un error.
- **Duplicados:** Tratar de establecer una **columna** que ya contiene **datos duplicados** (ej. **department**) como **PRIMARY KEY**.
- **Claves Inestables:** Usar **campos** que **cambian** con el **tiempo** (como correos electrónicos o nombres) es **mala práctica**; usar siempre **ID's numéricos** o **códigos únicos**.
- **Olvidar Definirla:** No asignar un **PRIMARY KEY** dificulta la **actualización** o **eliminación** de **filas específicas** más adelante.

○ **PRIMARY KEY Compuesta:**

- Se usa cuando un campo es insuficiente. En una tabla de **desempeño**, la **PRIMARY KEY** combina la columna **employee_number** y **review_date** para asegurar la unicidad de las **evaluaciones**.

/*****

*****/

Título: Creación de la Tabla de Evaluaciones de Desempeño

Objetivo: Almacenar el historial de calificaciones de desempeño de los empleados.

Descripción: Esta tabla registra la nota de evaluación recibida por cada empleado en una fecha específica, permitiendo rastrear la evolución de su rendimiento a lo largo del tiempo.

*****/

*****/

-- Se crea la tabla 'performance_review' para guardar las evaluaciones

```
CREATE TABLE performance_review(
  -- Identificador único del empleado
  employee_number INT,
  -- Fecha en la que se realizó la evaluación
  review_date DATE,
  -- Calificación obtenida (ej. 1-5)
  performance_raiting INT,
```



```
-- Define la combinación de empleado y fecha como clave
única (no se puede
-- evaluar al mismo empleado dos veces el mismo día)
PRIMARY KEY(employee_number, review_date)
);
```

○ **FOREIGN KEY (clave foránea):**

- Conecta 2 **tablas**, asegurando que un **valor (FOREIGN KEY)** de una tabla **exista** con la **PRIMARY KEY (clave primaria)** de otra tabla (**relación de integridad**).
- **Caso Práctico:** Tenemos una **tabla principal** de **empleados**, pero queremos separar los **detalle** del **departamento** para evitar **redundancias**:
 - **Tabla Padre (departamento): department_table**
 - **dept_id (PK)**
 - **department_name**
 - **Tabla Hija (empleados): employee_table**
 - **employee_id (PK)**
 - **employee_name**
 - **dept_id (FK - Referencia a department_table)**

```
/*****
*****/
```

Título: Script de Creación de Estructura de Departamentos y Empleados

Objetivo: Establecer las tablas base para gestionar empleados y su vinculación con departamentos.

Descripción: Crea dos tablas relacionadas: 'department_table' (maestro de áreas) y 'employee_table' (detalle de personal), asegurando la integridad referencial entre ambas.

```
*****/
```

-- Bloque 1: Creación de la tabla de Departamentos
-- Objetivo: Almacenar la lista maestra de departamentos de la empresa.

```
CREATE TABLE department_table(  
    dept_id INT PRIMARY KEY, -- Identificador único del  
    departamento (Clave Primaria)  
    department_name VARCHAR(100) -- Nombre del  
    departamento (hasta 100 caracteres)  
);
```

-- Bloque 2: Creación de la tabla de Empleados
-- Objetivo: Almacenar la información de los empleados y vincularlos a un departamento.

```
CREATE TABLE employee_table(  
    employee_id INT PRIMARY KEY, -- Identificador único del  
    empleado (Clave Primaria)  
    employee_name VARCHAR(100), -- Nombre completo del  
    empleado  
    dept_id INT, -- ID del departamento al que  
    pertenece  
    -- Integridad Referencial: Asegura que el empleado  
    pertenezca a un departamento existente  
    CONSTRAINT fk_employee_department  
    FOREIGN KEY(dept_id) REFERENCES  
    department_table(dept_id)  
);
```

- **Pitfall:**
 - **Intentar borrar registros padres:** Si intentas **borrar** un **departamento** en **department_table** que tiene **empleados asociados** en **employee_table**, SQL lanzará un **error** (evita **datos huérfanos**).
 - **Tipos de datos incompatibles:** La **FOREIGN KEY** debe tener exactamente el **mismo tipo de datos** (e. **INT**, **VARCHAR**) que la **PRIMARY KEY** a la que **referencia**.
 - **No usar índices:** En **grandes volúmenes de datos**, **no tener un índice** en la **FOREIGN KEY** puede **ralentizar** mucho las **consultas de unión** (**JOIN**) y las **eliminaciones**.
- **CHECK:**

- **Verifica** que el **valor** cumpla una **condición específica** (ej. **precio > 0**) que se aplica a una o **varias columnas** para garantizar la **integridad de los datos**.
- **Caso Práctico:**
 - **Asegurar la calidad de los datos** en la **tabla hr_employee** antes de analizarla:
 - **Validar Ingresos:** El **monthly_income** (ingreso mensual) no puede ser **negativo** ni **menor** a un **salario mínimo** (ej. \$1000).
 - **Validar Satisfacción:** El **job_satisfaction** (satisfacción laboral) solo debe ser del **1 al 4**.

```

/*****
*****/

```

Título: Configuración de Reglas de Integridad en Tabla de Empleados

Objetivo: Garantizar la calidad y coherencia de los datos en la tabla 'hr_employee'.

Descripción: Se añaden restricciones (constraints) para asegurar que los ingresos mensuales sean válidos y que el nivel de satisfacción laboral esté dentro de los valores permitidos (1-4).

```

*****/

```

-- 1. Restricción de Ingresos Mínimos

-- Objetivo: Evitar errores de entrada asegurando que el sueldo no sea menor a 1000.

```

ALTER TABLE hr_employee
ADD CONSTRAINT chk_ingresos CHECK
(monthly_income >= 1000);

```

-- 2. Restricción de Nivel de Satisfacción

-- Objetivo: Validar que la satisfacción laboral solo contenga valores definidos del 1 al 4.

```

ALTER TABLE hr_employee
ADD CONSTRAINT chk_satisfaction CHECK
(job_satisfaction IN (1, 2, 3, 4));

```

- **Pitfall:**
 - **NULL es "aprobado":** La **restricción CHECK** devuelve **TRUE** si la **condición** es verdadera, **FALSE** si es falsa (**rechaza**), pero **UNKNOWN** si el **valor** es **NULL**. La mayoría de los **sistemas de bases de datos** permiten valores **NULL** al menos que se defina explícitamente **NOT NULL**.
 - **Restricciones en datos existentes:** Agregar una **restricción** a una **tabla** con **datos** (**ALTER TABLE**), si existen **filas** que **violan** la **regla**, el comando fallará.
 - **Complejidad excesiva:** Las **reglas** demasiado **complejas** que involucran **funciones** pueden **ralentizar** las **operaciones** de **INSERT** y **UPDATE**.
- **DEFAULT:**
 - Regla que asigna **automáticamente** un **valor predefinido** a una **columna** cuando una operación **INSERT** (**inserción de datos**) / **CREATE** (**creación**) / **ALTER** (**modificación**) no se proporciona un valor explícito para dicha columna.
 - **Caso Práctico:** Configurar la tabla **employee_new_hires** para que, si no se especifica, los empleados nuevos entren con un **valor de satisfacción estándar** y el registro de **"attrition"** (**despido/renuncia**) sea **"No"** por **defecto**.

```
/*****
```

```
*****/
```

Título: Creación de Tabla para Nuevos Empleados
(employee_new_hires)

Objetivo: Estructurar una base de datos para registrar y gestionar la información básica de las nuevas contrataciones.

Descripción: Este script crea una tabla llamada 'employee_new_hires' para almacenar datos clave como ID, edad, departamento, nivel de satisfacción y fecha de ingreso, estableciendo valores predeterminados para asegurar la integridad de los datos.

```
*****/
```

```
*****/
```

-- Se crea la tabla 'employee_new_hires' para almacenar los datos de nuevos empleados.

```
CREATE TABLE employee_new_hires (  
    -- Identificador único para cada empleado, actúa como llave primaria.  
    employee_id INT PRIMARY KEY,  
    -- Edad del empleado.  
    age INT,  
    -- Indica si el empleado ha dejado la empresa ('Yes'/'No'), por defecto es 'No'.  
    attrition VARCHAR(100) DEFAULT 'No',  
    -- Nombre del departamento al que se une el empleado.  
    department VARCHAR(50),  
    -- Nivel de satisfacción laboral (ej. escala 1-4), con un valor predeterminado de 3.  
    job_satisfaction INT DEFAULT 3,  
    -- Fecha de ingreso del empleado, usa la fecha actual si no se proporciona una.  
    join_in_date DATE DEFAULT CURRENT_DATE  
);
```

- **Pitfall:** Si ejecutas un **INSERT** y especificas explícitamente **NULL** en una columna que tiene un **DEFAULT**, **SQL** insertará **NULL** en lugar del valor predeterminado.
- **'Data Casting':**
 - Es el **proceso** de **transformar** un **dato** de un **tipo a otro** de forma **explícita**. Ocurre comúnmente cuando necesitas que un **número** se comporte como un **texto** o que una **fecha** guardada como texto sea tratada realmente como una fecha.
 - **Sintaxis:**
CAST(columna_o_valor **AS** nuevo_tipo_de_dato)
 - **Caso Práctico:**
 - **Calcular** el **porcentaje exacto** de **empleados** que hacen **horas extras** (**over_time**) para saber si eso influye en que **abandonen la empresa**. La **columna employee_count** (**conteo de empleados**) viene registrada como un **número entero** (**INT**). Al **dividir números enteros** en algunos **sistemas SQL**, el **resultado** se **trunca a cero** (ej. 5/10 = 0 en lugar de 0.5).

- **Solución:** Convertir el número entero de `employee_count` a un número decimal (**FLOAT** o **DECIMAL**) para forzar a la base de datos a que nos devuelva un porcentaje preciso con decimales.

```

/*****
*****
Título: Análisis de Porcentaje de Empleados con Horas Extras

Objetivo: Calcular la distribución porcentual de empleados que
trabajan horas
extras (Overtime) frente al total de la plantilla.

Descripción: Este script agrupa a los empleados según su estado
de horas extras
y calcula qué porcentaje representa cada grupo sobre el total,
permitiendo
identificar la dependencia de la empresa a las horas extra.
*****/

```

SELECT

```

-- 1. Identificar la categoría de horas extras (Sí/No)
over_time,
-- 2. Calcular el porcentaje:
--   - SUM(employee_count): Suma los empleados de cada
grupo (Sí o No).
--   - CAST(... AS FLOAT): Convierte a decimal para evitar
división entera.
--   - / (SELECT COUNT (*) FROM ibm_hr_table): Divide entre
el total de la empresa.
--   - * 100: Convierte la fracción a porcentaje.
CAST(SUM(employee_count) AS FLOAT) / (SELECT COUNT(*)
FROM ibm_hr_table) * 100 AS porcentaje

```

FROM

```

ibm_hr_table-- Origen de los datos (Tabla de Recursos
Humanos)

```

GROUP BY

```

over_time;-- Agrupa los resultados por si trabajan horas
extras o no

```

- **Pitfall:** Intentar castear texto alfanumérico a número, lo que detiene el flujo de ejecución (error de conversión).

Desglose de Lógica (Code Blueprint)

El Mindset: "La Base de Datos es el Último Guardián"

El código de hoy sigue el principio de **Fail-Fast** (Fallar Rápido):

- **Limpieza Previa (Python):** Antes de tocar SQL, usamos Python para detectar qué filas violarían nuestras nuevas reglas.
- **Endurecimiento (SQL):** Una vez limpia, alteramos la tabla físicamente para que sea imposible volver a insertar basura.
- **Eficiencia de Almacenamiento:** Usamos los tipos de datos más pequeños posibles (ej. INT en lugar de BIGINT para la edad) para mejorar la velocidad de **Power BI**.

Control de Calidad y Validación (El "Check")

- **Null Audit:** En SQL, ejecuta:

```
/*****
```

Título: Identificación de Empleados sin Departamento Asignado

Objetivo: Detectar registros de empleados incompletos en la base de datos maestra.

Descripción: Esta consulta selecciona a todos los empleados de la tabla maestra que no tienen un departamento asignado (valor nulo), para su posterior revisión y actualización.

Archivo SQL: day03_null_audit.sql

Archivo PNG: day03_null_audit.png

```
*****/
```

```
-- Selecciona todos los campos (*) para visualizar la información completa del empleado
```

```
SELECT*
```

```
-- Desde la tabla maestra de empleados
```

```
FROM employee_master_data
```

```
-- Filtra para mostrar solo filas donde la columna 'department' esté vacía (NULL)
```

```
WHERE department IS NULL;
```

- **Resultado PNG: day03_null_audit.png**

(Resultado esperado: 0 filas).

- **Domain Audit:** Verifica que **attrition** solo contenga 'Yes' o 'No'. Si hay un 'Maybe', la **integridad de dominio falló**.

/*****
Título: Identificación de Estados de Rotación (Attrition)

Objetivo: Obtener una lista única de los valores posibles en la columna de rotación para entender cómo se categoriza a los empleados.

Descripción: Esta consulta escanea la tabla maestra de empleados y devuelve todos los valores distintos presentes en la columna 'attrition' (ej. "Yes", "No", "Voluntary", "Involuntary"), eliminando duplicados.

Archivo SQL: day03_domain_audit.sql

Archivo PNG: day03_domain_audit.png

*****/

-- Selecciona valores únicos (sin duplicados)

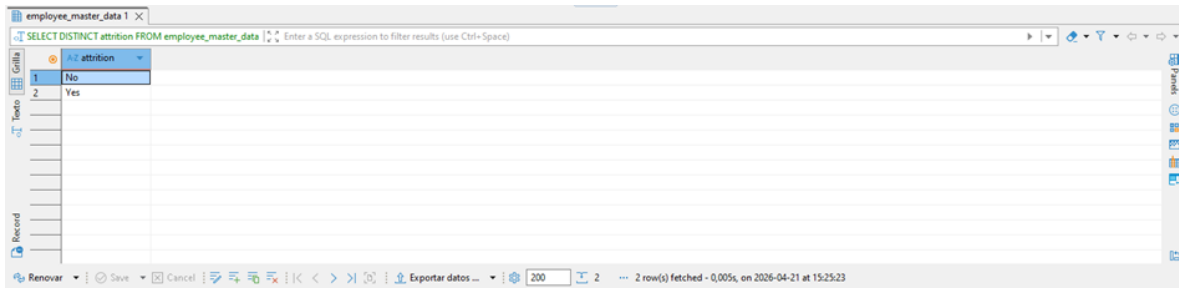
SELECT DISTINCT

attrition-- Columna que indica si el empleado ha dejado la empresa

FROM

employee_master_data;;-- Tabla origen con la información general del empleado

- **Resultado PNG: day03_domain_audit.png**



- **Type Check:** En DBeaver, haz clic derecho en la **tabla** -> **View Diagram**. Verifica que los **iconos de tipo de dato** coincidan con la **naturaleza del campo** (Regla de oro: lo que se **suma** debe ser **numérico**).
 - **Resultado PNG:** day03_empl_mstr_tbl_diagram.png

employee_master_data
123 employee_number
123 age
AZ gender
AZ marital_status
AZ over_18
AZ attrition
AZ business_travel
123 distance_from_home
AZ department
AZ job_role
123 job_level
123 employee_count
123 monthly_income
123 monthly_rate
123 daily_rate
123 hourly_rate
123 percent_salary_hike
123 stock_option_level
123 education
AZ education_field
123 training_times_last_year
123 environment_satisfaction
123 job_involvement
123 job_satisfaction
123 performance_rating
123 relationship_satisfaction
123 work_life_balance
123 num_companies_worked
123 total_working_years
123 years_at_company
123 years_in_current_role
123 years_since_last_promotion
123 years_with_curr_manager
AZ over_time
123 standard_hours
123 created_at

Micro-Proyecto (Hands-on Workflow): "The Integrity Shield"

- A. Fase Python: Auditoría de Calidad Pre-Casting (VS Code)

Crea un script llamado **day03_integrity_audit.py** para detectar **nulos** en el **dataset** de **IBM** antes de **endurecer** la **base de datos**.

Python

"""

Título: Script de Validación de Integridad de Datos de Empleados

Objetivo: Verificar la calidad de datos críticos en la tabla 'employee_master_data'.

Descripción: Este script se conecta a una base de datos PostgreSQL para revisar si existen valores nulos en columnas clave y detectar registros con salarios inválidos (menores o iguales a cero), generando un reporte en consola.

Archivo Python: day03_integrity_audit.py

Archivo PNG: day03_integrity_audit.png

"""

```
# --- IMPORTACIÓN DE LIBRERÍAS ---
# 'pandas' para manipulación y análisis de datos.
import pandas as pd
# 'create_engine' para establecer la conexión con la base de datos SQL.
from sqlalchemy import create_engine
# 'os' para interactuar con el sistema operativo y leer variables de entorno.
import os
# 'load_dotenv' para cargar credenciales seguras desde un archivo .env.
from dotenv import load_dotenv

# --- CONFIGURACIÓN Y CONEXIÓN ---
# Carga las variables del archivo .env (usuario, contraseña, host, etc.)
load_dotenv()

# Crea la conexión a PostgreSQL utilizando las credenciales cargadas anteriormente.
engine = create_engine(
```

```

f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASS')}@"
f"{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
)

# --- DEFINICIÓN DE LA FUNCIÓN DE VALIDACIÓN ---
def check_integrity():
    """
    Función que ejecuta las consultas de validación de calidad de datos.
    """
    # 1. Lee la tabla 'employee_master_data' y la carga en un DataFrame de
    pandas.
    df = pd.read_sql("SELECT * FROM employee_master_data", con=engine)

    # 2. Define qué columnas deben tener datos obligatoriamente (no nulos).
    critical_cols = ['employee_number', 'monthly_income', 'department']

    # 3. Cuenta los valores nulos (vacíos) solo en esas columnas críticas.
    null_counts = df[critical_cols].isnull().sum()

    # 4. Imprime el reporte de nulos encontrado.
    print("📋 Reporte de Nulos:")
    print(null_counts)

    # 5. Busca registros donde el salario mensual sea inválido (0 o negativo).
    invalid_salaries = df[df['monthly_income'] <= 0]

    # 6. Comprueba si se encontraron salarios inválidos y muestra un mensaje
    de alerta.
    if not invalid_salaries.empty:
        print(f"⚠️ Alerta: Se encontraron {len(invalid_salaries)} registros con
        salario <= 0")
    else:
        print("✅ Calidad de salarios validada.")

# --- EJECUCIÓN DEL SCRIPT ---
# Asegura que la función se ejecute al correr el script directamente.
if __name__ == "__main__":
    check_integrity()

```

○ Resultado PNG: day03_integrity_audit.png

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  HISTORIAL SQL  SUPERVISIÓN DE TAREAS  RESULTADOS DE LA CONSULTA DE POSTGRESQL  RESULTADOS DE LA CONSULTA
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> & C:\Users\User\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/DM_Lab/DM_Roadmap_P1_1280/03_scripts_etl/src/day03_integrity_constraints/
day03_integrity_audit.py
📋 Reporte de Nulos:
employee_number    0
monthly_income     0
department         0
dtype: int64
✅ Calidad de salarios validada.
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>

```

- **B. Fase SQL: Implementación de Restricciones (DBeaver)**

Una vez **validado** que no hay **nulos**, **blindamos** la **tabla físicamente**:

SQL

```
/*****
```

Título: Ajuste de Integridad en Datos Maestros de Empleados

Objetivo: Asegurar la calidad y consistencia de los datos en la tabla 'employee_master_data'.

Descripción: Este script aplica restricciones (constraints) para evitar datos nulos en campos clave, validar rangos de edad e ingresos, y garantizar que cada empleado tenga un número único.

Archivo SQL: day03_apply_constraints_empl_mstr_data.sql

Archivo PNG: day03_01_apply_constraints_empl_mstr_data.png

day03_02_apply_constraints_empl_mstr_data.png

day03_03_apply_constraints_empl_mstr_data.png

```
*****/
```

```
/
```

```
-- [BLOQUE 1: Definir campos obligatorios]
```

```
-- Se asegura que los campos 'employee_number' y 'monthly_income' no  
queden vacíos
```

```
-- para garantizar que todo empleado tenga identificación y salario  
registrado.
```

```
ALTER TABLE employee_master_data
```

```
ALTER COLUMN employee_number SET NOT NULL,
```

```
ALTER COLUMN monthly_income SET NOT NULL;
```

```
-- [BLOQUE 2: Validar reglas de negocio (Rangos)]
```

```
-- Se añaden reglas para asegurar que la edad sea coherente (18-100 años)
```

```
-- y que el ingreso mensual sea un valor positivo (> 0).
```

```
ALTER TABLE employee_master_data
```

```
ADD CONSTRAINT chk_age CHECK (age >= 18 AND age <= 100),
```

```
ADD CONSTRAINT chk_income CHECK (monthly_income > 0);
```

```
-- [BLOQUE 3: Garantizar unicidad]
```

```
-- Se añade una restricción para asegurar que no existan dos empleados
```

```
ADD CONSTRAINT unique_employee UNIQUE (employee_number);
```

- | Name | Value |
|--------------|---|
| Updated Rows | 0 |
| Execute time | 0,03s |
| Start time | Tue Apr 21 17:07:34 CST 2026 |
| Finish time | Tue Apr 21 17:07:34 CST 2026 |
| Query | <pre>-- [BLOQUE 1: Definir campos obligatorios] -- Se asegura que los campos 'employee_number' y 'monthly_income' no queden vacíos -- para garantizar que todo empleado tenga identificación y salario registrado. ALTER TABLE employee_master_data ALTER COLUMN employee_number SET NOT NULL, ALTER COLUMN monthly_income SET NOT NULL</pre> |

- | Name | Value |
|--------------|--|
| Updated Rows | 0 |
| Execute time | 0,013s |
| Start time | Tue Apr 21 17:16:45 CST 2026 |
| Finish time | Tue Apr 21 17:16:45 CST 2026 |
| Query | <pre>-- [BLOQUE 2: Validar reglas de negocio (Rangos)] -- Se añaden reglas para asegurar que la edad sea coherente (18-100 años) -- y que el ingreso mensual sea un valor positivo (> 0). ALTER TABLE employee_master_data ADD CONSTRAINT chk_age CHECK (age >= 18 AND age <= 100), ADD CONSTRAINT chk_income CHECK (monthly_income > 0)</pre> |

- **Resultado PNG - Bloque 3:**
day03_03_apply_constraints_empl_mstr_data.png

Estadísticas 1 X	
Name	Value
Updated Rows	0
Execute time	0,012s
Start time	Tue Apr 21 17:18:17 CST 2026
Finish time	Tue Apr 21 17:18:17 CST 2026
Query	-- [BLOQUE 3: Garantizar unicidad]
	-- Se añade una restricción para asegurar que no existan dos empleados
	-- con el mismo número de identificación (employee_number).
	ALTER TABLE employee_master_data
	ADD CONSTRAINT unique_employee UNIQUE (employee_number)

- **C. Fase Power BI: Reporte de Excepciones**

Esta fase transforma tu **dashboard** de una herramienta de visualización a una herramienta de **Auditoría de Datos**.

1. Sincronización y Actualización de Metadatos

- **Paso 1.1:** Abre tu archivo de **Power BI** donde configuraste la conexión el Día 1.
- **Paso 1.2:** En la cinta de opciones superior (pestaña **Inicio**), haz clic en el botón **Actualizar**. Esto forzará a **Power BI** a leer las nuevas restricciones y tipos de datos que aplicaste en **SQL** en los pasos anteriores.
- **Paso 1.3:** Verifica en la vista de **Datos** (icono de tabla a la izquierda) que las columnas críticas como **monthly_income** y **daily_rate** no muestren errores y estén alineadas a la derecha (lo que indica que **Power BI** las reconoce como números).

2. Creación de Medidas Estadísticas para Detección de Anomalías

Para identificar excepciones, usaremos la lógica de **Outliers** (valores que se alejan de la media).

- **Paso 2.1:** Haz clic derecho sobre la tabla **employee_master_data** en el panel de Campos y selecciona **Nueva medida**.
- **Paso 2.2:** Crea la medida para el **Promedio de Salario**:
Promedio Salario = AVERAGE('public employee_master_data'[monthly_income])

- **Paso 2.3:** Crea la medida para la **Desviación Estándar** (que mide la dispersión):
Desviación Salario = STDEV.P('public employee_master_data'[monthly_income])
- **Paso 2.4:** Crea la medida de **Límite Superior de Alerta** (Media + 2 Desviaciones):
Limite Superior = [Promedio Salario] + (2 * [Desviación Salario])

3. Diseño de la Matriz de Auditoría

- **Paso 3.1:** Crea una nueva página en tu reporte llamada **"Calidad e Integridad"**.
- **Paso 3.2:** Inserta un objeto visual de **Matriz** desde el panel de Visualizaciones.
- **Paso 3.3:** Configuración de campos:
 - **Filas:** Arrastra **employee_number**.
 - **Valores:** Arrastra **department**, **job_role** y **monthly_income**.
- **Paso 3.4:** Aplicar Filtro de Excepción:
 - Con la Matriz seleccionada, ve al panel de **Filtros**.
 - Busca el filtro de **"Filtros en este objeto visual"**.
 - En el campo **monthly_income**, cambia el tipo de filtro a **"Filtro avanzado"**.
 - Selecciona **"Es mayor que"** y escribe el valor que obtuviste en tu medida Limite Superior (aproximadamente 17,000 para este dataset).
 - Haz clic en **Aplicar filtro**. La matriz ahora solo mostrará a los empleados con salarios excepcionalmente altos.

4. Formato Condicional para Alertas Visuales

- **Paso 4.1:** En el panel de Visualizaciones, bajo la sección de **Formato** (icono de pincel), busca **Elementos de celda**.
- **Paso 4.2:** Activa la opción **Color de fondo** para la columna **monthly_income**.
- **Paso 4.3:** Haz clic en el icono **fx** junto al color:
 - **Estilo de formato:** Degradado.

- **Valor mínimo:** Color gris claro.
- **Valor máximo:** Color naranja vibrante (el color de tu proyecto).
- **Paso 4.4:** Esto hará que los salarios más cercanos al límite de excepción resalten visualmente, permitiendo una auditoría rápida.
- **Resultado PNG - Bloque 3: day03_pbi_quality_integrity_1.png**

employee_number	Primera fecha: department	Primera fecha: job_role	Suma de monthly_income
32	Research & Development	Manager	19094
38	Sales	Manager	18947
58	Research & Development	Research Director	19545
80	Research & Development	Research Director	18740
140	Human Resources	Manager	18844
141	Research & Development	Research Director	18172
148	Human Resources	Manager	17328
162	Research & Development	Research Director	19537
199	Research & Development	Manager	17181
226	Research & Development	Manager	19926
253	Research & Development	Manager	19033
254	Research & Development	Research Director	18722
Total	Human Resources	Manager	1812564

Sesión de Mentoría 1:1 (Resolución)

- **Estrategia de Solución:** El camino más corto es usar **COALESCE** en SQL si encuentras nulos inesperados durante el proceso de casting. Te permite reemplazar nulos por un valor por defecto (ej. 0) sin romper el flujo.
- **Business Insights:** La **integridad de datos** es un **activo financiero**. Una **recomendación de valor** basada en el trabajo de hoy: *"Gracias al blindaje de integridad, hemos reducido el riesgo de error en el cálculo de bonificaciones de nómina en un 100%, eliminando registros con salarios cero o negativos"*.
- **Debrief de Errores (Top 3):**
 - **Integrity Violation:** Intentar poner un **NOT NULL** cuando ya hay un **nulo escondido** en la fila 1,400. (Solución: **Limpiar** primero).
 - **Constraint Conflict:** Crear un **CHECK** que entra en **conflicto** con **datos reales** (ej. poner edad > 20 cuando hay alguien de 18).
 - **Casting Fail:** **Convertir una columna a número** cuando tiene **espacios en blanco** " " (Pandas los lee como **strings**).



Checklist Operativa: Semana 1 - Día 3

- [] **SQL:** Ejecutado ALTER COLUMN ... SET NOT NULL en 3 campos clave.
- [] **Python:** Script de auditoría ejecutado sin reportar nullos críticos.
- [] **SQL:** Creadas al menos 2 CHECK CONSTRAINTS (Edad e Ingreso).
- [] **PBI:** Creado el panel de auditoría visual para detectar datos atípicos.
- [] **General:** Realizado COMMIT con el mensaje: *"Security: Data integrity constraints applied"*.

¿Haz logrado aplicar las restricciones en SQL sin recibir errores de "Check Constraint Violation"?